

GroupMemBench

GroupMemBench 笔记

论文: (<https://arxiv.org/abs/2605.14498>)

作者: Jingbo Yang、Kwei-Herng Lai、Xiaowen Wang、Shiyu Chang、Yaar Harari、
Evgeniy Gabrilovich
机构: UC Santa Barbara、Microsoft

代码: [UCSB-NLP-Chang/GroupMemBench](#)

数据: [kimperyang/GroupMemBench](#)

一、motivation

1. 现有 Agent 记忆主要为单人对话设计

现有方法大多面向单用户对话，因此通常将历史看成一条按时间排列的文本流，不需要特别判断“谁在说话、谁在提问”

真实工作场景却包含多名用户、多个帖子和回复线程。同一内容由不同人说出，可能对应不同偏好或记忆更新；同一问题由不同用户提出，也可能需要不同答案。因此，单人记忆方法不能直接处理群体记忆

2. 多人群聊的三类困难

2.1 记忆系统能否理解线程、多人回复和跨话题关系，而不是把群聊当成一串独立消息？

群聊包含新帖、线程回复、争论、共识形成和跨项目交流。`reply_to` 关系说明一条消息在回应哪条消息，以及它是在支持、反驳还是补充已有观点，如果只按时间拼接消息，系统可能保留文本，却丢失讨论结构

2.2 同一问题由不同人提出，答案可能不同，所以系统能否记清谁说了什么、谁在提问？

例如Alice负责市场材料，Bob负责负载测试。两人都问“发布前我还需要做什么”，正确答案应当不同

2.3 不同角色会用不同词表达同一件事，系统是否要能跨表达找到并理解正确证据？

例如工程师可能说 具体工程问题，管理者可能只说对齐工作要求。系统不能只匹配表面词语，还要知道这些表达在当前团队语境中是否指向同一件事

二、方法

图结构保存：

1.场景设定

初始图包含四类静态节点：

- **Domain（项目）**：表示一个对话。类别标签（category tags），类别标签用于判断不同对话是否相关
- **Topic（主题）**：保存 d::topic_label 格式，避免不同对话中的同名主题发生冲突
- **Phase（阶段）**：具有时间范围的子问题，包含进展类别、状态目标、开始日期、目标日期、阶段负责人和虚拟文档链接
- **User（用户）**：表示对话参与者，包含角色、语气、表达风格和专业程度

四类静态节点之间的关系：

```
Domain → Topic → Phase
User ↔ Domain
User ↔ Phase
Domain ↔ related_to_project ↔ Domain
```

Message：消息正文、作者、时间戳和生成路径类型
生成消息后动态加入

```
Message → authored_by → User
Message → posted_in → Phase
Message → reply_to → Parent Message
```





2.逐条生成群聊消息

系统不是一次生成整段群聊，而是每轮完成：

采样交流路径

→ 组装当前上下文

→ GPT-5生成一条消息

→ 将消息写回图中

2.1基于路径的交流场景采样

这一阶段的目的
每生成一条消息前，系统先在知识图中采样一条路径
系统以 (0.20:0.60:0.20) 的比例采样三种交流场景：

场景	路径	作用
新帖	Domain → Topic → Phase → User	开启一个讨论线程
线程回复	Domain → Topic → Phase → Parent Message → User	回复已有消息
跨项目交流	User → Phase → Domain → Related Domain → Phase → User	将一个项目的经验带到相关项目

路径采样同时确定目标Phase、作者、父消息或目标用户。选择时偏向尚未覆盖的项目、活跃或无人回复的Phase以及较新的父消息；跨项目路径还要求时间方向合理，并优先连接相同角色的用户

(程序代码实现)

该阶段得到：

路径类型：post reply cross_project_repl

路径	输出节点
新帖	Domain、Topic、Phase、作者User
回复	Domain、Topic、Phase、Parent Message、回复者User
跨项目	源User、源Phase、源Domain、目标Domain、目标Phase、目标User

一组带有相关信息的节点，确定“谁在什么Phase中发帖或回复谁

2.2 组装当前消息的情境上下文

输入确定的路径，系统围绕目标Phase构造当前上下文：

该阶段得到

- S_u
- 目标Phase最近最多50条消息
 - 可选的3个较早历史片段
 - 当前Active Sub-issue
 - Phase Ledger
 - 25%概率加入的兄弟Phase信息

提示词为：

每个Phase第一次使用时，GPT-5生成6个互不重叠的Sub-issues，用于引导后续讨论

输入：

[当前Phase的信息；具体字段和序列化方式未公开]

任务：

请列出 K=6 个不同且互不重叠的子问题。

这些子问题应当是真实的跨职能团队在当前Phase中可能讨论的问题。

输出：

6个子问题``

每新增8条消息，系统根据最近16条消息调用GPT-5更新一次不超过180词的Ledger：

Decided: 已经确定的事项
Open Questions: 尚未解决的问题
Pending Commitments: 已经承诺但尚未完成的工作

提示词为：

输入：
[当前Phase最近的消息，窗口最多16条]
[是否输入旧Ledger、如何排列字段：论文未明确公开]

任务：
将当前Phase的讨论情况重写为一个不超过180词的摘要。

输出必须包含三个部分：

DECIDED
已经确定的事项。

OPEN QUESTIONS
尚未解决的问题，并记录问题重复出现的次数。

PENDING COMMITMENTS
已经作出但尚未完成的承诺。

如果某个**OPEN**问题已经在新消息中得到回答，
必须将其从**OPEN QUESTIONS**迁移到**DECIDED**。

最后，系统以0.50为序列匹配阈值删除高度相似的上下文片段，减少机械重复
场景图构建完成后，系统不会一次性生成整段群聊，而是每轮只生成一条消息，再把它写回图中

2.3 基于情境上下文生成消息

输入	内容
(S_u)	2.2组装的情境上下文
(C_u)	发言者画像：角色、语气、风格、专业程度
stage	Phase当前处于 early、middle 或 late
path	post、reply 或 cross-project
Phase信息	Topic、Phase名称、状态目标、目标日期
Parent Message	仅回复场景需要

程序根据路径选择对应Prompt模板，
例如普通回复prompt模板：

Background: 作者和Phase背景

Phase: 阶段、状态目标和日期

Channel history: 情境上下文 S_u

Parent message: 要回复的父消息

Task: 回应父消息并推动讨论

Voice: 作者的语气、风格和专业程度

Constraints: 长度和措辞要求

并有一定概率加入噪声消息：

机制	概率	作用
Noise	约5%	加入细微错误、旧信息或跑题内容
Disagreement	回复的25%	对已有观点提出实质性异议
Style imperfection	约20%	加入错字、残句或自我修正
Ultra-short	回复的30%	生成1—6个词的短回复
Tone jitter	10%—20%	轻微改变用户语气
Decision change	符合条件Phase的约5%	中途推翻已有决定

具体加入哪类噪声由程序选择，GPT-5只负责按照给定条件生成文本。生成结果被保存为Message节点，建立链接，写回图中：

Message --authored_by--> User

Message --posted_in----> Phase

Message --reply_to-----> Parent Message

:

2.4合成群聊质量评估

这一阶段不是继续生成消息，而是检查前面生成的整批对话是否足够像真实工作群聊。

评估对象

作者比较三类对话：

① 图结构引导的合成群聊

└─ 本文方法生成的四个领域数据

- ② Single-Prompt Baseline
 - └ 不使用知识图，直接用一个大Prompt生成群聊
- ③ Real-World Upper Bound
 - └ 作者收集的一个月真实群聊记录

使用什么评估？

作者采用经过调整的 **G-Eval**方法，让GPT-5充当评审模型

每个维度评分范围是：

$s_k \in [0, 5]$

最终平均分为：

$S_{avg} = \frac{1}{6} \sum_{k=1}^6 s_k$

六个评价维度

维度	实际检查什么
Naturalness	消息读起来是否像真人工作群聊
Coherence	前后消息是否连接自然、逻辑一致
Diversity	内容、表达风格和参与者是否足够多样
Contextual Relevance	消息是否与项目、Phase及前文相关
Momentum	对话是否推动问题解决，而不是原地重复
Engagingness	不同参与者是否真正互动、回应和追问

3.构造对抗性评测问题

① Anchor Selection：选择锚点消息

程序先用轻量规则查找包含目标实体或事实的候选消息

例如：

“Carol：供应商A的交付时间改成下周一。”

然后由LLM判断：

这条消息是否包含一个明确、可以提问并回答的事实？

不明确的消息会被排除

② Initial Composition：生成初始问题

一个针对特定问题类型的LLM，根据：

- 锚点消息；
- 问题类型
- 标准答案

LLM自动输出证据、确定候选答案

问题类型	测试能力
Multi-Hop Reasoning	串联多条消息才能得到答案
Knowledge Update	识别最新信息，避免回答旧信息
Term Ambiguity	根据提问者身份理解歧义词
User-Implicit Reasoning	理解“我”“我的”等第一人称指代
Temporal Reasoning	根据时间戳和时间表达进行推理
Abstention	对群聊中没有的信息拒绝回答

生成初始问题

锚点：交付时间改成下周一

目标答案：下周一

问题：供应商A现在什么时候交付？

③ Solve–Judge–Refine：解答—判断—改写循环

这是最重要的一步：

初始问题

↓

基线Agent检索群聊并回答

↓

LLM Judge检查回答

如果基线Agent答对：

说明问题太简单

→ LLM继续改写，让问题更困难

如果基线Agent答错：

说明问题已经足够困难

→ 接受为Benchmark题目

这里的设计目标是：**保留能够难住一个有能力的检索基线的问题**

④ **Output：保存最终题目**

每道接受的问题保存：

question	最终问题
gold_answer	标准答案
asking_user_id	谁在提问
evidence_trace	每轮使用的证据记录

三、实验

主实验

Table 1. Performance of various methods on the Multi-Hop Question Answering dataset.

Method	Multi-Hop	Update	Ambiguity	Implicit	Temporal	Abstention	Average
RAG-based Methods							
BM25	<u>40.11</u>	<u>25.23</u>	14.15	40.82	54.94	<u>77.98</u>	<u>43.22</u>
text-embedding-3-large	36.26	23.36	21.70	46.94	<u>32.72</u>	75.23	38.04
GraphRAG [32]	12.09	14.02	19.81	14.29	5.56	66.97	20.56
Agent Memory Systems							
Mem0 [9]	21.98	4.67	11.32	20.41	16.67	82.57	25.73
MemGPT [28]	22.53	17.76	20.75	28.57	12.42	<u>77.98</u>	28.15
A-Mem [29]	35.16	22.43	26.42	46.94	23.46	67.89	35.10
HippoRAG [19]	39.56	27.10	<u>30.19</u>	<u>42.86</u>	29.63	75.23	39.72
Hindsight [8]	42.31	17.76	37.74	40.82	54.94	77.06	46.01

实验环节	使用模型	作用
记忆写入（Ingestion）	GPT-4o-mini	抽取事实、总结消息、构建记忆节点或知识图
向量化与稠密检索	text-embedding-3-large	将消息或记忆转成向量，计算语义相似度
检索后的问答	GPT-5	根据各基线检索出的记忆回答问题
准确性裁判	GPT-5	比较预测答案和标准答案，输出正确或错误

模块	输入	输出
GPT-5问答Agent	提问用户、问题、Top-10检索消息	推理文本+最终答案

模块	输入	输出
GPT-5 Judge	问题、标准答案、Agent最终答案	判分理由 + Correct/Incorrect

GPT-4o-mini作用:

把原始群聊消息交给记忆系统，让系统把它们加工成以后可以检索和使用的长期记忆
例如对于不同的基线方法

- BM25：保存原文，建立关键词索引
- Embedding：把原文转换成向量
- Mem0：抽取成“交付时间=下周一”
- A-MEM：写成记忆笔记并建立关联
- GraphRAG：提取实体和关系
- Hindsight：将消息整合成经验记忆

问题类型	测试能力
Multi-Hop Reasoning	串联多条消息才能得到答案
Knowledge Update	识别最新信息，避免回答旧信息
Term Ambiguity	根据提问者身份理解歧义词
User-Implicit Reasoning	理解“我”“我的”等第一人称指代
Temporal Reasoning	根据时间戳和时间表达进行推理
Abstention	对群聊中没有的信息拒绝回答

性能—效率权衡气泡图

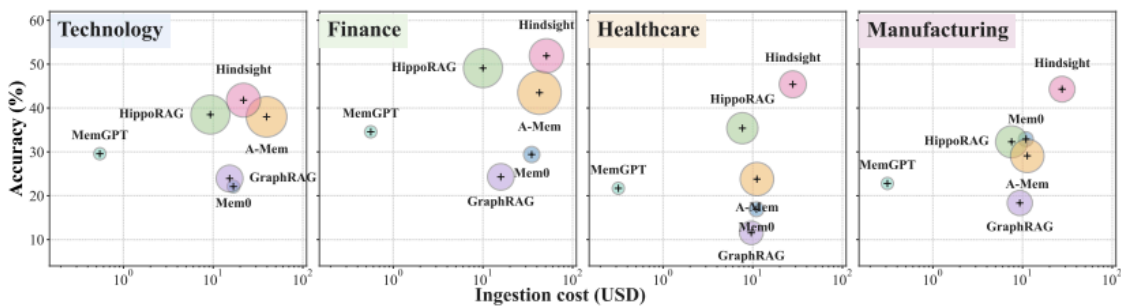
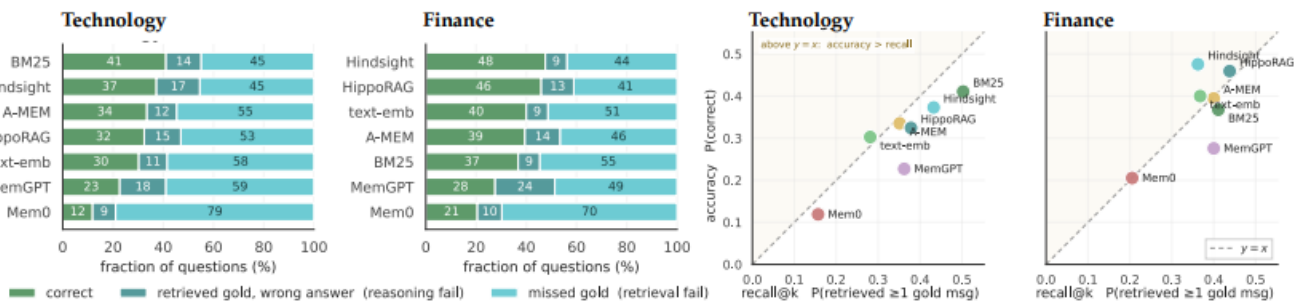


Figure 4: Performance–efficiency trade-off across the four domains. Each marker is one of six baselines (five agent memory systems plus GRAPH-RAG): x -axis is ingestion cost (USD, log scale), y -axis is per-domain QA accuracy (%), and bubble area encodes on-disk storage in GB. Per-(baseline, domain) numerical values are listed in Table 11.

图中元素	含义
四个子图	Technology、Finance、Healthcare、Manufacturing四个领域
横轴	建立记忆的写入成本，单位为美元
纵轴	该领域的问答准确率
气泡大小	记忆库占用的磁盘空间
一个气泡	一种记忆系统

失败模式分解与检索召回率—回答准确率分析



- #问题
- 1.问题精炼会不会把题从"难"改成"病态", 不符合正常难度, 以致最终分数偏低?
 - 2.拒答是因为确实没有检索到还是模型检索到了但是能力不够
强模型会不会即使检索不到也编造, 弱模型能力弱检索不到反而高
 - 3.未来信息泄漏问题
BM25基线测评时, 讲对话全部输入之后再进行的提问, 如果遇到时间推理类问题, 会不会影响准确性